Kubernetes Core Objects

pods, Deployments, Services, Ingress

Watch the 4-min video

In one line: you declare desired state as objects (a **Deployment** for replicated pods, a **Service** for a stable address, an **Ingress** for HTTP routing, ConfigMaps and Secrets for configuration) and controllers continuously reconcile reality towards it; **labels and selectors** tie the objects together.

1 What you need after docker run

You've containerised the FastAPI app (lesson 55). In production you need three copies, restarted when they crash, spread over machines, updated without downtime, reachable at a stable internal address, and exposed on `api.shop.com` with TLS. Kubernetes provides each of these with a small set of object types.

2 Declarative state and the cluster

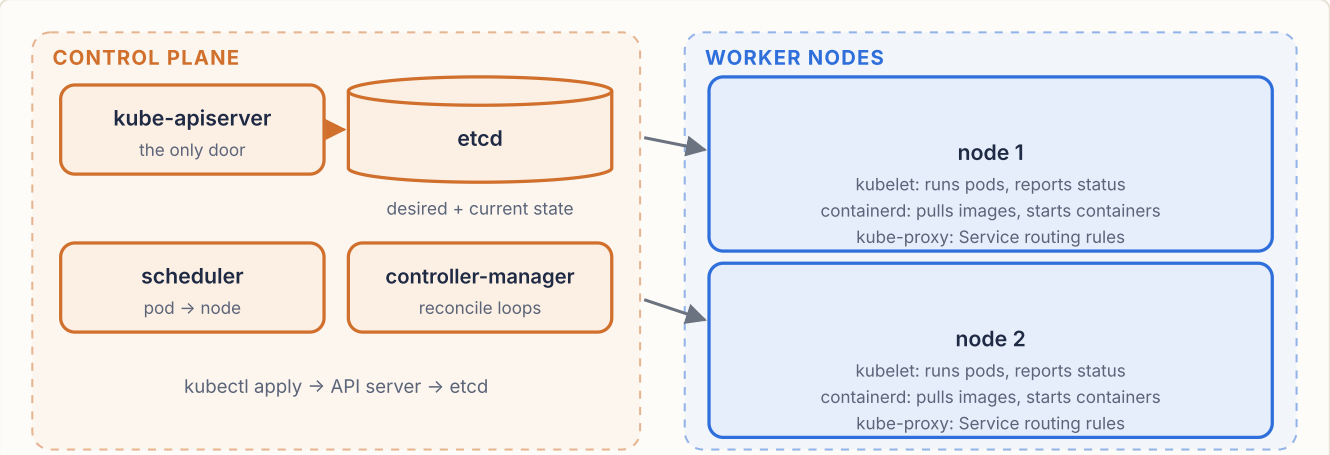


Figure 1. Everything goes through the API server and is stored in etcd; controllers and kubelets watch it and act.

You never tell Kubernetes "start a container"; you submit YAML saying what should exist. The **API server** validates and stores it in **etcd**. **Controllers** in the controller manager run reconcile loops: observe current state, compare with desired, act to close the gap. The **scheduler** assigns new pods to nodes (lesson 57). On each node, the **kubelet** starts the pod's containers through the container runtime and reports status. Because everything is a loop, the system self-heals: delete a pod and a controller simply creates a new one.

3 Pods: the disposable unit

A **pod** is one or more containers that share a network namespace (one IP, talking over localhost) and volumes, always scheduled together on one node. Typical multi-container pods have a main container plus a sidecar (log shipper, proxy). Pods are **disposable**: when a node dies, its pods are gone, and replacements get new names and new IPs elsewhere. So you almost never create bare pods; you create a controller that manages them.

4 Deployments and ReplicaSets

A **Deployment** declares "run 3 replicas of this pod template". It creates a **ReplicaSet**, whose only job is to keep exactly that many matching pods running. When you change the template (a new image tag), the Deployment creates a new ReplicaSet and performs a **rolling update**: scale the new one up and the old one down step by step, by default with `maxSurge: 25%` and `maxUnavailable: 25%`. A new pod counts only when it passes its **readiness probe**, so a broken release stalls instead of replacing healthy pods. Old ReplicaSets are kept (10 by default), so `kubectl rollout undo` is a fast rollback.

Probes matter: **readiness** decides whether a pod receives traffic; **liveness** restarts a stuck container; **startup** protects slow starters from liveness kills. For stateful workloads, a **StatefulSet** gives stable names and volumes (lesson 59); for one pod per node, a **DaemonSet**; for run-to-completion work, a **Job** or **CronJob**.

5 Services: a stable address

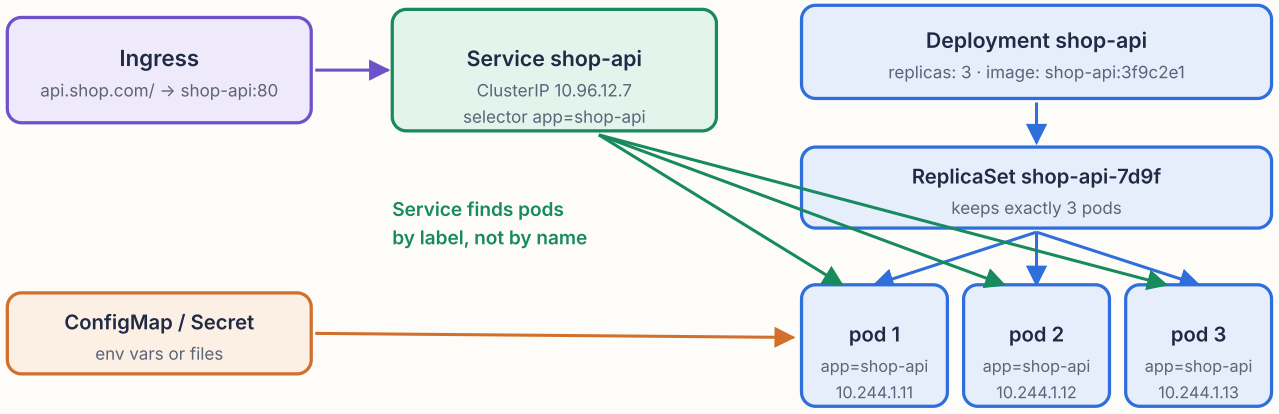


Figure 2. Labels are the glue: the Deployment owns pods labelled app=shop-api; the Service routes to whatever ready pods carry that label. All live in namespace shop.

A **Service** gives a changing set of pods one stable virtual IP and DNS name, `shop-api.shop.svc.cluster.local` (or just `shop-api` from the same namespace). Its **selector** picks pods by label; Kubernetes keeps an **EndpointSlice** listing the ready pod IPs, and kube-proxy (iptables or IPVS rules on every node, or an eBPF data plane such as Cilium) load-balances connections to them. Types: **ClusterIP** (internal only, the default), **NodePort** (a port in 30000–32767 on every node), **LoadBalancer** (asks the cloud for an external load balancer), and **headless** (no virtual IP; DNS returns the pod IPs, used by StatefulSets). Networking details are in lesson 60.

6 Ingress: HTTP from outside

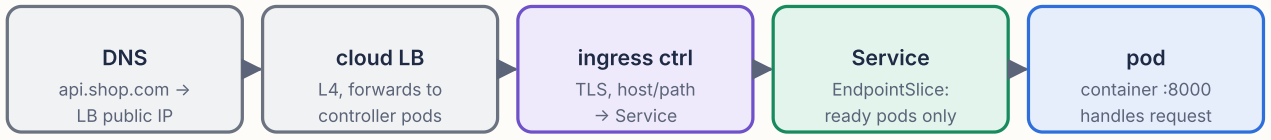


Figure 3. The request path. Many controllers proxy straight to pod IPs from the EndpointSlices.

An **Ingress** is a set of HTTP routing rules: host `api.shop.com`, path `/` → Service `shop-api` port 80, with TLS from a Secret. The Ingress object does nothing by itself; an **ingress controller** (NGINX-based, Traefik, HAProxy, or a cloud one such as Azure Application Gateway for Containers or the AWS Load Balancer Controller) watches Ingress objects and configures a real proxy, usually behind one cloud load

balancer. The newer **Gateway API** (GA since 2023) splits this into Gateway and HTTPRoute objects with richer routing, and is the direction new clusters are moving.

7 ConfigMaps, Secrets and namespaces

ConfigMaps hold non-sensitive configuration (up to 1 MiB each); **Secrets** hold passwords, keys and certificates. Both are mounted as environment variables or files; files update in place when the object changes, environment variables only on restart. Secrets are only **base64-encoded**, not encrypted, by default: enable encryption at rest for etcd, restrict them with RBAC, or sync them from a real vault (External Secrets Operator, Secrets Store CSI driver). **Namespaces** group objects per team or environment, and carry their own RBAC rules, ResourceQuotas and network policies.

```
apiVersion: apps/v1                    apiVersion: v1
kind: Deployment                       kind: Service
metadata: {name: shop-api}             metadata: {name: shop-api}
spec:                                  spec:
  replicas: 3                            selector: {app: shop-api}
  selector:                              ports: [{port: 80, targetPort: 8000}]
    matchLabels: {app: shop-api}       ---
  template:                            apiVersion: networking.k8s.io/v1
    metadata:                          kind: Ingress
      labels: {app: shop-api}          metadata: {name: shop-api}
    spec:                              spec:
      containers:                        tls: [{hosts: [api.shop.com],
      - name: api                               secretName: shop-tls}]
        image: shop-api:3f9c2e1          rules:
        ports: [{containerPort: 8000}]   - host: api.shop.com
        readinessProbe:                    http:
          httpGet: {path: /health,           paths:
                    port: 8000}              - path: /
        envFrom:                               pathType: Prefix
        - configMapRef: {name: shop-cfg}       backend: {service:
                                                 {name: shop-api, port: {number: 80}}}
```

Figure 4. The worked example as YAML: Deployment (left), Service and Ingress (right). The labels and the port numbers must line up.

WORKED EXAMPLE: DEPLOYMENT + SERVICE + INGRESS FOR FASTAPI

The Deployment runs 3 replicas of `shop-api:3f9c2e1`, each with a readiness probe on `/health` and config from the `shop-cfg` ConfigMap. The ClusterIP Service `shop-api` selects `app=shop-api` and maps port 80 to container port 8000. The Ingress routes `api.shop.com` to that Service with the `shop-tls` certificate. Changing the image to `shop-api:8b1d0aa` starts a rolling update: one new pod is added, becomes ready, enters the Service's endpoints, one old pod is removed, and so on, with zero downtime. If the new pods never become ready, the rollout stalls with the old pods still serving.

COMMON MISTAKES

- Creating bare pods, which nobody recreates when a node dies.
- No readiness probe, so traffic hits pods that are still starting.
- A Service selector that doesn't match the pod labels: the Service has no endpoints.
- Believing Secrets are encrypted because they look scrambled (base64).

8 Key terms

Pod: co-scheduled containers sharing an IP; disposable

Deployment: desired replicas + rolling updates

ReplicaSet: keeps N matching pods running

Service: stable virtual IP and DNS name for pods

Ingress: HTTP host/path routing, implemented by a controller

Labels / selectors: how objects find each other

9 Interview questions

Q1. Explain the path of a request from Ingress to a pod. BASIC

The domain resolves to the ingress controller's load balancer; the controller matches host and path rules and forwards to the Service; the Service's virtual IP load-balances to a ready pod matching its selector, whose container handles it.

Q2. Why not create pods directly? BASIC

Pods are disposable; Deployments keep the desired count running through a ReplicaSet and handle rolling updates and rollbacks.

Q3. What's the difference between readiness and liveness probes? BASIC

Readiness controls whether the pod gets traffic; liveness restarts the container when it's stuck.

Q4. A Service returns connection errors though the pods are running. What do you check? SCENARIO

Whether its selector matches the pods' labels and the pods are ready (kubectl get endpointslices), and whether targetPort matches the container port.

Q5. A new release crashes on start. What does the Deployment do? SCENARIO

New pods never become ready, so with maxUnavailable limits the rollout stalls while old pods keep serving; you roll back with kubectl rollout undo.

Q6. Does Ingress traffic always pass through the Service's ClusterIP? TRICKY

No: many ingress controllers read the EndpointSlices and proxy directly to pod IPs; the Service is used for discovery.

Q7. Are Kubernetes Secrets secure by default? TRICKY

They're only base64-encoded in etcd; you need encryption at rest, tight RBAC, or an external secret store.

Next: SD-V-007 · Kubernetes Scheduling backendroadmap.com